Path: Client/client_handlers.py

```
# client_handlers.py
# í ·º …ì ·ì ,í , ì ·º†/í , í ,º /º ‹ º“¤º

import os
import json
import sys
sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from tkinter import messagebox, filedialog, END, DISABLED, NORMAL
from tkinter import LabelFrame, Label, scrolledtext
from env_utils import DEFAULT_HOST, DEFAULT_PORT
from Client.client_core import TCPClient
from ai_services import OPENAI_AVAILABLE, OPENAI_ERROR, generate_keywords, generate_summary, generat


class ClientHandlers:
    """í ·º …ì ·ì ,í , ì ·º†/í , í ,º /º ‹ í ·º  ì /"""

    def __init__(self, ui, tcp_client):
        self.ui = ui
        self.tcp_client = tcp_client
        self.current_user_email = None
        self.current_file_id = None
        self.current_file_data = None
        self.current_quiz_data = None

        # ì ·º†/í , º° ì ,º '
        self._bind_events()

    def _bind_events(self):
        """UI ì ·º†/í , º° ì ,º '"""
        # º¡ Œ•,ì , ì ·º†/í ,
        if hasattr(self.ui, 'login_btn'):
            self.ui.login_btn.config(command=self.handle_login)
        if hasattr(self.ui, 'logout_btn'):
            self.ui.logout_btn.config(command=self.handle_logout)

        # ì  º†  ì °Œ†° º† í … ì  Œ–° (ì  º   ì °Œ†°)
        self.ui.convert_btn.config(command=self.show_convert_form)
        self.ui.folder_btn.config(command=self.show_folder_form)
        self.ui.select_file_btn.config(command=self.select_audio_file)
        self.ui.convert_btn_main.config(command=self.handle_convert)
        self.ui.folder_tab_add.config(command=lambda: self.switch_folder_tab('add'))
        self.ui.folder_tab_update.config(command=lambda: self.switch_folder_tab('update'))
        self.ui.folder_tab_delete.config(command=lambda: self.switch_folder_tab('delete'))
        self.ui.folder_add_btn.config(command=self.handle_folder_add)
        self.ui.folder_update_btn.config(command=self.handle_folder_update)
        self.ui.folder_delete_btn.config(command=self.handle_folder_delete)
        self.ui.summary_btn.config(command=lambda: self.show_content('summary'))
        self.ui.quiz_btn.config(command=lambda: self.show_content('quiz'))
        self.ui.score_btn.config(command=self.score_quiz)
        self.ui.add_quiz_btn.config(command=self.add_quiz_question)
        self.ui.project_tree.tag_bind('file', '<Double-1>', self.on_file_select)

    def handle_login(self):
```

```
        """ì ¬ìš©ì ë¡ê·¸ì¸ ì²ë¦¬ (Folder í ì ·º, ì ì  í ì ,)"""
        username = self.ui.login_username_entry.get().strip()
        email = self.ui.login_email_entry.get().strip()
        password = self.ui.login_password_entry.get().strip()

        # ì ·º' ì …ì  í ì
        if not email:
            self.ui.login_result_label.config(text="ì ·º' ì …ì  ì  º ¥í ·ì£…ì ,ì  .", fg='red')
            return

        # ì ‹ì 'ì  º“ ì · ì  ì …º'· ì ·º' ì …ì  ì  ì¶ ì¶
        if not username:
            username = email.split('@')[0]

        # º„ ºº º† í ,Œ°  ì  ì …º'· º„  º‹,ì  ì ·º¡  ì † ºƒ‹ (ì  í  ì ‹í ›)
        if not password:
            password = ""

        # ì  º† ì  ì °Œ†°º  ì · ì  º  ì§  í  ì ,
        if not self.tcp_client.is_connected:
            self.ui.login_result_label.config(text="º¤…ì  ì  º† ì  ì °Œ†°í ·ì£…ì ,ì  .", fg='red')
            return

        # ì  º† ì  º¡ Œ•,ì  , ì  ì †› (Folder í  ì ·º, ì ì  í  ì ,)
        result = self.tcp_client.send_command("login", {
            "username": username,
            "email": email,
            "password": password
        })

        if result.get("status") == "success":
            user = result.get("user", {})
            self.current_user_email = email
            self.ui.current_user_email = email
            self.ui.current_username = user.get("user_name", username)
            self.ui.is_logged_in = True

            # º¡ Œ•,ì  , ì°‰ º «Œ,°
            self.ui.login_window.destroy()

            # º' ì  , ì°‰ í  ì
            self.ui.root.deiconify()

            # º¡ Œ•,ì  , ì  í  ì  º °ì ·í ,
            self.ui.login_status_label.config(text=f"º¡ Œ•,ì ,: {email}", fg='green')
            self.ui.logout_btn.pack(fill=X, pady=2)

            messagebox.showinfo("ì –Œ‡µ", result.get("message", "º¡ Œ•,ì  , ì –Œ‡µ"))
        else:
            self.ui.login_result_label.config(
                text=result.get("message", "º¡ Œ•,ì  , ì ⁄í ¤"), fg='red'
            )

    def handle_logout(self):
        """º¡ Œ•,ì  ì  ì † ºƒ‹"""
```

```
        self.current_user_email = None
        self.ui.is_logged_in = False

        # º¡ Œ•,ì , ì  í   ì  º °ì ·í ,
        self.ui.login_status_label.config(text="º¡ Œ•,ì , í  ì  ", fg='#f44336')
        self.ui.logout_btn.pack_forget()

        # º¡ Œ•,ì , ì°‰ º /ì   í  ì
        self.ui.create_login_window()

        messagebox.showinfo("ì  ºƒ…", "º¡ Œ•,ì  ì  º  ì  ì µº  º /.")

    def connect_server(self):
        """ì  º†  ì °Œ†°"""
        host = self.ui.server_host_entry.get().strip() or DEFAULT_HOST
        try:
            port = int(self.ui.server_port_entry.get().strip() or DEFAULT_PORT)
        except ValueError:
            messagebox.showerror("ì /º¥ ", "í ‹í , º† í ,º  ì «ì  ì ‹ì … í '°  º /.")
            return

        if self.tcp_client.connect(host, port):
            self.ui.server_status_label.config(text=f"ì °Œ†° ì  í  : ì °Œ†°º ¤ ({host}:{port})", fg=
            messagebox.showinfo("ì –Œ‡µ", "ì  º† ì  ì °Œ†°º  ì  ì µº  º /.")
        else:
            self.ui.server_status_label.config(text="ì °Œ†° ì  í  : ì °Œ†° ì /í ¤", fg='red')
            messagebox.showerror("ì /º¥ ", "ì  º† ì  ì °Œ†°í  ì  ì  ì µº  º /.")

    def disconnect_server(self):
        """ì  º†  ì °Œ†° í ·ì  """
        self.tcp_client.disconnect()
        self.ui.server_status_label.config(text="ì °Œ†° ì  í  : ºfl,ì °Œ†°", fg='#666')
        messagebox.showinfo("ì  ºƒ…", "ì  º†  ì °Œ†°ì · í ·ì  º  ì  ì µº  º /.")

    def show_convert_form(self):
        """º‡ í  í … í  ì  """
        self.hide_all_forms()
        self.ui.convert_frame.pack(fill='x', pady=10)
        self.ui.document_header.pack_forget()
        self.ui.index_frame.pack_forget()

    def show_folder_form(self):
        """í ·º  Œ· ºƒ‹ í … í  ì  """
        self.hide_all_forms()
        self.ui.folder_frame.pack(fill='x', pady=10)
        self.ui.document_header.pack_forget()
        self.ui.index_frame.pack_forget()
        self.switch_folder_tab('add')

    def switch_folder_tab(self, tab):
        """í ·º  Œ· ºƒ‹ í › ì  í  """
        # º“¤º  í › º† í … ºƒ‹ì
        self.ui.folder_tab_add.config(bg='#f0f0f0', fg='#666')
        self.ui.folder_tab_update.config(bg='#f0f0f0', fg='#666')
```

```
        self.ui.folder_tab_help.config(bg='#f0f0f0', fg='#666')

        # º"¤º  ì „ì  ì ¤Œ,°Œ,°
        self.ui.folder_add_section.pack_forget()
        self.ui.folder_update_section.pack_forget()
        self.ui.folder_delete_section.pack_forget()

        # ì  í  í  í › í  ì –í
        if tab == 'add':
            self.ui.folder_tab_add.config(bg='#333333', fg='white')
            self.ui.folder_add_section.pack(fill='x', pady=10)
        elif tab == 'update':
            self.ui.folder_tab_update.config(bg='#333333', fg='white')
            self.ui.folder_update_section.pack(fill='x', pady=10)
        elif tab == 'delete':
            self.ui.folder_tab_delete.config(bg='#333333', fg='white')
            self.ui.folder_delete_section.pack(fill='x', pady=10)

    def show_content(self, content_type):
        """ì‰ í  ì ,  í  ì  """
        self.hide_all_forms()

        if content_type == 'summary':
            self.ui.summary_btn.config(bg='#333333', fg='white')
            self.ui.quiz_btn.config(bg='#f0f0f0', fg='#666')
            self.ui.summary_frame.pack(fill='both', expand=True, pady=10)
            self.ui.quiz_frame.pack_forget()
        elif content_type == 'quiz':
            self.ui.summary_btn.config(bg='#f0f0f0', fg='#666')
            self.ui.quiz_btn.config(bg='#333333', fg='white')
            self.ui.summary_frame.pack_forget()
            self.ui.quiz_frame.pack(fill='both', expand=True, pady=10)
            # í ·ìƒ  í ›ì   º  º¥·º  ì  Œ°  OpenAIì  í ·ìƒ  ì  ì†› (í  ì  ì  )
            self.load_quiz_if_needed()

    def hide_all_forms(self):
        """º"¤º  í … ì ¤Œ,°Œ,°"""
        self.ui.convert_frame.pack_forget()
        self.ui.folder_frame.pack_forget()
        self.ui.summary_frame.pack_forget()
        self.ui.quiz_frame.pack_forget()

    def load_file_list(self):
        """ì  º† ì  ì  í  ì … º"'º¡  º¡ º  í  ì ‹ í  º¡ ì  í  , í  ,ºƒ‹ì  í  ì  """
        if not self.tcp_client.is_connected:
            return

        try:
            result = self.tcp_client.send_command("list_files", {})
            if result.get("status") == "success":
                files = result.get("files", [])
                # Œ,°ì¡· í  ,ºƒ‹ í  ›º"' ì  Œ–° (º£¤í  , ì  ì  ,)
                for item in self.ui.project_tree.get_children(self.ui.project_root):
                    self.ui.project_tree.delete(item)
```

```
                for file_data in files:
                    file_id = str(file_data.get('file_id'))
                    title = file_data.get('title', f'í ì … {file_id}')
                    self.ui.project_tree.insert(
                        self.ui.project_root, 'end',
                        text=title,
                        tags=('file', file_id)
                    )
            except Exception as e:
                print(f"í ì … º"'º¡ º¡ º  ì /º¥ : {e}")

        def on_file_select(self, event):
            """í ì … ì  í  ì ·º†/í ,"""
            item = self.ui.project_tree.selection()[0]
            tags = self.ui.project_tree.item(item, 'tags')
            if 'file' in tags:
                file_id = tags[1] if len(tags) > 1 else None
                if file_id:
                    self.load_file(file_id)

        def load_file(self, file_id):
            """í ì … º¡ º  """
            # º‹,ì  í /º  ì  ì ,º –ì / º† í … í  ì
            self.ui.document_header.pack(fill='x', pady=(0, 20))
            self.ui.index_frame.pack(fill='x', pady=(0, 20))

            # í ì … ì  º"' ì  º º¡ ·í ,
            item = None
            for i in self.ui.project_tree.get_children():
                for j in self.ui.project_tree.get_children(i):
                    for k in self.ui.project_tree.get_children(j):
                        if 'file' in self.ui.project_tree.item(k, 'tags'):
                            if self.ui.project_tree.item(k, 'tags')[1] == file_id:
                                item = k
                                break

            if item:
                title = self.ui.project_tree.item(item, 'text')
                self.ui.document_title.config(text=title)

            # í ì … º‡ Œ†‰ ì  ì ·ì  í ·ìƒ º º¡ ·í ° ì· Œ,°í
            if self.current_file_id != int(file_id):
                self.current_quiz_data = None
                if hasattr(self.ui, 'quiz_frames'):
                    self.build_quiz_ui({"questions": []})

            # ì º† ì  ì  í ì … º ·ì ' º¡ º
            if self.tcp_client.is_connected:
                result = self.tcp_client.send_command("load_file", {"file_id": int(file_id)})
                if result.get("status") == "success" and result.get("file_data"):
                    self.current_file_id = int(file_id)
                    self.current_file_data = result["file_data"]
                    self.update_file_content(self.current_file_data)
```

```
                quiz_data = self.current_file_data.get("quiz")
                if quiz_data and quiz_data.get("questions"):
                    # quiz_idì   user_answerº¥… í ‹í ¤í  º  º¡  º‡ í
                    quiz_data_with_ids = {
                        "questions": [
                            {
                                "question": q.get("question", ""),
                                "answer": q.get("answer", ""),
                                "explanation": q.get("explanation", ""),
                                "quiz_id": q.get("quiz_id"),
                                "user_answer": q.get("user_answer", "")  # ì ·ì   º µì   í ‹í ¤
                            }
                            for q in quiz_data.get("questions", [])
                        ]
                    }
                    self.current_quiz_data = quiz_data_with_ids
                    # í ·ìƒ UI ì  º °ì ·í , (í ·ìƒ í ›ì · º‡·ì ·ì§ ì  ì  º  º °ì ·í °º  ì⁄ º„ )
                    if hasattr(self.ui, 'quiz_frames'):
                        self.build_quiz_ui(quiz_data_with_ids)
                else:
                    # í ·º „ í  ì …ì  í ·ìƒ Œ° ì  ì …º'· º„ ì  í  º¡ ì· Œ,°í
                    self.current_quiz_data = {"questions": []}
                    if hasattr(self.ui, 'quiz_frames'):
                        self.build_quiz_ui({"questions": []})

            # Œ,°º‡,ì  ì …º¡ ì  ì ‰ ì „ì  í  ì
            self.show_content('summary')

        def update_file_content(self, file_data):
            """í  ì … º ·ì  ' ì  º °ì ·í ,"""
            if file_data.get("keywords"):
                self.ui.keyword_label.config(text=file_data["keywords"])
            if file_data.get("summary"):
                self.ui.summary_label.config(text=file_data["summary"])
            if file_data.get("text"):
                self.ui.conversion_text.config(state=NORMAL)
                self.ui.conversion_text.delete('1.0', END)
                self.ui.conversion_text.insert('1.0', file_data["text"])
                self.ui.conversion_text.config(state=DISABLED)
            # í ·ìƒ  º °ì ·í °º   load_fileì  ì   ì† ºƒ‹í  ºfl º¡  ì ‹Œ,°ì  º  ì  º °ì ·í ,í  ì§ ì  ì

        def select_audio_file(self):
            """ì ⁄º  ì ⁄ í  ì … ì  í  """
            file_path = filedialog.askopenfilename(
                title="ì ⁄º  ì ⁄ í  ì … ì  í  ",
                filetypes=[("ì ⁄º  ì ⁄ í  ì …", "*.mp3 *.wav *.m4a *.ogg *.flac"), ("º“¤º  í  ì …", "*.
            )
            if file_path:
                self.selected_audio_file = file_path
                filename = os.path.basename(file_path)
                self.ui.audio_file_label.config(text=f"ì  í  º  í  ì …: {filename}")

        def handle_convert(self):
            """ì ⁄º  ì ⁄ º‡ í  ì† ºƒ‹ (í ·º …ì ·ì ,í ,ì  ì   OpenAI STT ì§ ì  ì† ºƒ‹)"""
```

```
        if not hasattr(self, 'selected_audio_file') or not self.selected_audio_file:
            messagebox.showwarning("Œ†‰Œ‡ ", "í  ì …ì  ì  í  í ·ì£…ì ,ì  .")
            return

        # í ·º …ì ·ì ,í ,ì  ì  OpenAI STT º‡ í  ì † ºƒ‹
        try:
            from ai_services import STT_AVAILABLE, STT_ERROR, transcribe_audio

            if not STT_AVAILABLE:
                error_msg = "OpenAI APIº¥… ì ‹ì 'í  ì  ì  ì µº  º /."
                if STT_ERROR:
                    error_msg += f"\nì /º¥  ì  ì ,: {STT_ERROR}"
                error_msg += "\n\nº /ì  ì  í  ì ,í ·ì£…ì ,ì  :"
                error_msg += "\n1. OpenAI API í /Œ°  .env í  ì …ì  ì /ì  º  ì · ì  º  ì§  í  ì ,"
                error_msg += "\n2. openai í ¤í /ì§ Œ°  ì /ì„ º  ì · ì  º  ì§  í  ì ,: pip install op
                messagebox.showerror("STT ì /º¥ ", error_msg)
                return

            # 1º ¤Œ‡ : OpenAI STT º‡ í  (í ·º …ì ·ì ,í ,ì  ì  ì§ ì  ì † ºƒ‹)
            filename = os.path.basename(self.selected_audio_file)
            self.ui.convert_result_label.config(text="ð  OpenAI STT º‡ í  ì/ ... (ì  Œ° ì · Œ–,ºƒ
            self.ui.root.update()

            try:
                text = transcribe_audio(self.selected_audio_file, language="ko")
            except Exception as e:
                error_msg = f"STT º‡ í  ì /í ¤: {str(e)}"
                self.ui.convert_result_label.config(text=f"â  {error_msg}", fg='red')
                messagebox.showerror("STT ì /º¥ ", error_msg)
                return

            if not text:
                messagebox.showwarning("Œ†‰Œ‡ ", "º‡ í  º  í  ì /í ,Œ°  ì  ì µº  º /.")
                return

            # º‡ í  º  í  ì /í ,º¥… UIì  í  ì
            self.ui.conversion_text.config(state=NORMAL)
            self.ui.conversion_text.delete('1.0', END)
            self.ui.conversion_text.insert('1.0', text)
            self.ui.conversion_text.config(state=DISABLED)

            # 2º ¤Œ‡ : OpenAIº¡  ì  ì ‰/í /ì  º  ì  ì – (í ·º …ì ·ì ,í ,ì  ì  ì§ ì  í ,ì¶ )
            summary_text = ""
            keyword_text = ""
            if OPENAI_AVAILABLE:
                try:
                    self.ui.convert_result_label.config(text="ð /  OpenAIº¡  í /ì  º  /ì  ì ‰ ì  ì –
                    self.ui.root.update()

                    # í /ì  º
                    keyword_text = generate_keywords(text)

                    # ì  ì ‰
                    summary_text = generate_summary(text)
```

```
                if keyword_text:
                    self.ui.keyword_label.config(text=keyword_text)
                if summary_text:
                    self.ui.summary_label.config(text=summary_text)

            except Exception as e:
                # OpenAI ì /º¥ º   STT Œ†°Œ‡… ì ‹ì 'ì  º  ì  í ¥ ì  Œ† º§  ì † º ƒ‹
                self.ui.convert_result_label.config(
                    text=f"STT ì  º£ , OpenAI ì  ì ‰/í /ì  º  ì  ì – ì /  ì /º¥ : {e}", fg='oran
                )

        # 3º ¤Œ‡ : ì  º† ì  í  ì /í ,/ì  ì ‰/í /ì  º  ì  ì ¥ ì  ì †› ºº  í  º¡ ì  í , Œ,°º¡  º°
        if not self.tcp_client.is_connected:
            # ì  º† ì °Œ†° ì ‹ì  º
            from env_utils import DEFAULT_HOST, DEFAULT_PORT
            if not self.tcp_client.connect(DEFAULT_HOST, DEFAULT_PORT):
                self.ui.convert_result_label.config(
                    text="â  º‡ í  ì  º£  (ì  º† ì °Œ†° ì /í ¤º¡  ì  ì ¥í  ì §  ì  ì  )",
                    fg='orange'
                )
                messagebox.showwarning("Œ†‰Œ‡ ", "ì  º† ì  ì °Œ†°í  ì  ì  · º °ì ·í °º¥… ì
                return

        self.ui.convert_result_label.config(text="ð  / DBì  ì  ì ¥ ì / ...", fg='blue')
        self.ui.root.update()

        save_result = self.tcp_client.send_command("save_text", {
            "filename": filename,
            "text": text,
            "keywords": keyword_text,
            "summary": summary_text
        })

        if save_result.get("status") == "success":
            self.ui.convert_result_label.config(text="â  º‡ í  ì  º£  ºº  DB ì  ì ¥ ì  º£ ", f
            # í  º¡ ì  í , Œ,°º¡  í ,º ƒ‹ ì  º °ì ·í , (ì  ì †· º〃'º¡  º /ì  º¡ º  )
            self.load_file_list()
            # í  ì ‹ í  ì … ID ì  ì ¥
            file_id = str(save_result.get("file_id"))
            if file_id:
                self.current_file_id = int(file_id)
        else:
            error_msg = save_result.get('message', 'ì  ì  º  ì /º¥ ')
            self.ui.convert_result_label.config(
                text=f"â  º‡ í  ì  º£  (DB ì  ì ¥ ì /í ¤: {error_msg})",
                fg='red'
            )
            messagebox.showerror("DB ì  ì ¥ ì /º¥ ", f"º °ì ·í °º† ì ·ì / ì  ì ¥ì  ì /í ¤í  ì µ

    except Exception as e:
        import traceback
        traceback.print_exc()
        self.ui.convert_result_label.config(text=f"â  STT/ì  ì ‰ ì † º ƒ‹ ì /º¥ : {e}", fg='red')
        messagebox.showerror("ì /º¥ ", f"STT/ì  ì ‰ ì † º ƒ‹ ì /  ì /º¥ Œº  ºº ì  í  ì µº  º /:\n{e
```

Client/client_handlers.py

```
    def handle_folder_add(self):
        """í .º   ì ¶ Œ° """
        if not self.current_user_email:
            messagebox.showerror("ì /º¥ ", "ì ‹ì 'ì   ì ·º' ì …ì · ì /ì  º  ì§  ì  ì  ì µº  º /. º¡
            return

        folder_name = self.ui.folder_name_add_entry.get().strip()
        if not folder_name:
            messagebox.showwarning("Œ†‰Œ‡ ", "í ·º   ì ·ºƒ ì   ì  º ¥í ·ì£…ì ,ì  .")
            return

        result = self.tcp_client.send_command("manage_folder", {
            "action": "create",
            "folder_name": folder_name,
            "user_email": self.current_user_email
        })

        if result.get("status") == "success":
            self.ui.folder_add_result.config(text=f"â   {result.get('message', 'í ·º  Œ°  ì ¶ Œ° º  ì
        else:
            self.ui.folder_add_result.config(text=f"â   {result.get('message', 'í ·º   ì ¶ Œ°  ì /í ¤

    def handle_folder_update(self):
        """í .º   ì  ì  """
        if not self.current_user_email:
            messagebox.showerror("ì /º¥ ", "ì ‹ì 'ì   ì ·º' ì …ì · ì /ì  º  ì§  ì  ì  ì µº  º /. º¡
            return

        old_name = self.ui.folder_name_old_entry.get().strip()
        new_name = self.ui.folder_name_new_entry.get().strip()

        if not old_name or not new_name:
            messagebox.showwarning("Œ†‰Œ‡ ", "Œ,°ì ¡· í ·º   ì ·ºƒ Œ‡… ì   í ·º   ì ·ºƒ ì   º“¤º   ì
            return

        result = self.tcp_client.send_command("manage_folder", {
            "action": "update",
            "folder_name": old_name,
            "user_email": self.current_user_email,
            "new_folder_name": new_name
        })

        if result.get("status") == "success":
            self.ui.folder_update_result.config(text=f"â   {result.get('message', 'í ·º  Œ°  ì  ì  º
        else:
            self.ui.folder_update_result.config(text=f"â   {result.get('message', 'í ·º   ì  ì   ì /

    def handle_folder_delete(self):
        """í .º   ì ›ì  """
        if not self.current_user_email:
            messagebox.showerror("ì /º¥ ", "ì ‹ì 'ì   ì ·º' ì …ì · ì /ì  º  ì§  ì  ì  ì µº  º /. º¡
            return

        folder_name = self.ui.folder_name_delete_entry.get().strip()
```

```
            if not folder_name:
                messagebox.showwarning("Œ†‰Œ‡ ", "í ·º  ì ·ºƒ ì  ì  º ¥í ·ì£…ì ,ì  .")
                return

            if not messagebox.askyesno("í  ì ,", f'"{folder_name}" í ·º  º¥… ì  º§  ì ›ì  í  ì  Œ† ì µº
                return

            result = self.tcp_client.send_command("manage_folder", {
                "action": "delete",
                "folder_name": folder_name,
                "user_email": self.current_user_email
            })

            if result.get("status") == "success":
                self.ui.folder_delete_result.config(text=f"â  {result.get('message', 'í ·º  Œ°  ì ›ì  º
            else:
                self.ui.folder_delete_result.config(text=f"â  {result.get('message', 'í ·º  ì ›ì  ì /

        def score_quiz(self):
            """í ·ìƒ  ì – ì  º°  º µì  DB ì  ì ¥"""
            if not getattr(self.ui, "quiz_frames", None):
                messagebox.showinfo("ì  º…", "ì – ì  í  í ·ìƒ Œ°  ì  ì µº  º /.")
                return

            if not self.current_file_id:
                messagebox.showwarning("Œ†‰Œ‡ ", "í  ì …ì · ì  í  º  ì§  ì  ì  ì µº  º /.")
                return

            correct_count = 0
            total_count = len(self.ui.quiz_frames)
            quiz_results = []  # DB ì  ì ¥ì '

            for frame in self.ui.quiz_frames:
                user_answer = frame.input_box.get('1.0', END).strip()
                correct_answer = getattr(frame, "correct_answer", "").strip()
                explanation = getattr(frame, "explanation", "")
                quiz_id = getattr(frame, "quiz_id", None)

                # Œ° º ¤í  í ‹í ¤ ì ‹º¶  Œ,°ì /  º„ Œµ  (Œ‡µ°°–/º  ì  º‹,ì  º‹·ì  )
                ua_norm = user_answer.replace(" ", "").lower()
                ca_norm = correct_answer.replace(" ", "").lower()

                correct_is = False
                if ua_norm and ca_norm and ca_norm in ua_norm:
                    frame.result_label.config(text="â  ì  º µì  º  º /!", fg='green')
                    correct_count += 1
                    correct_is = True
                else:
                    msg = f"â  ì /º µì  º  º /.\nì  º µ: {correct_answer}"
                    if explanation:
                        msg += f"\ní ·ì /: {explanation}"
                    frame.result_label.config(text=msg, fg='red')

                # DB ì  ì ¥ì ' º °ì ·í ° ì  ì §
                if quiz_id:
```

```
                quiz_results.append({
                    "quiz_id": quiz_id,
                    "user_answer": user_answer,
                    "correct_is": correct_is
                })

        if total_count > 0:
            percentage = int((correct_count / total_count) * 100)
            self.ui.final_score_label.config(
                text=f"ð  fl ì µ ì¢  ì  ì  : {correct_count} / {total_count} º‹,ì   ({percentage}ì  )"
            )
            self.ui.final_score_label.pack(pady=10)

        # ì  º† ì  Œ°  í ·ì ƒ ì  º µì  ì  ì ¥
        if self.tcp_client.is_connected and quiz_results:
            result = self.tcp_client.send_command("save_score", {
                "file_id": self.current_file_id,
                "quiz_results": quiz_results
            })
            if result.get("status") == "success":
                # ì  ì ¥ ì –Œ‡µ í  current_quiz_data ì  º °ì ·í , (º µì  º° ì  )
                for i, q in enumerate(self.current_quiz_data.get("questions", [])):
                    for quiz_result in quiz_results:
                        if q.get("quiz_id") == quiz_result.get("quiz_id"):
                            self.current_quiz_data["questions"][i]["user_answer"] = quiz_result.get(
                            break

    def load_quiz_if_needed(self):
        """í  ì ‹ í  ì …ì  º  í · DBì  ì  í ·ì ƒ º¡ º  (í ·ì ƒ í ›ì  ì · º  í ,ì ¶ )"""
        if not self.current_file_id:
            return

        # ì ·ºfl, º¡ º  º  í ·ì ƒ º °ì ·í °Œ°  ì  Œ‡ í  ì ‹ í  ì …Œ‡… ì …ì „ í  º'· Œ•,Œ† ì  ì ‹ì '
        if self.current_quiz_data and self.current_quiz_data.get("questions"):
            self.build_quiz_ui(self.current_quiz_data)
            return

        # ì  º† ì  ì  í  ì ‹ file_idì  í ·º „í  º  í ·ì ƒ º§  º¡ º  (º /º¥, í  ì …ì  í ·ì ƒ Œ°  í
        if self.tcp_client.is_connected:
            result = self.tcp_client.send_command("load_quizzes", {"file_id": self.current_file_id})
            if result.get("status") == "success":
                quizzes = result.get("quizzes", [])
                if quizzes:
                    # DBì  ì  º¡ º  í  í ·ì ƒ º¥… UIì  í  ì  (file_idº¡ í  í °º§ º  í ·ì ƒ º§ ,
                    quiz_data = {
                        "questions": [
                            {
                                "question": q.get("question", ""),
                                "answer": q.get("correct_answer", ""),
                                "explanation": q.get("explanation", ""),
                                "quiz_id": q.get("quiz_id"),
                                "user_answer": q.get("user_answer", "")  # ì ·ì  º µì  í ‹í ¤
                            }
                            for q in quizzes
                        ]
```

```
                    self.current_quiz_data = quiz_data
                    self.build_quiz_ui(quiz_data)
                    return

        # í ·ìƒ Œ° ì ì …°'· °„ ì í °¡ í ì
        self.current_quiz_data = {"questions": []}
        self.build_quiz_ui({"questions": []})

    def add_quiz_question(self):
        """í ·ìƒ í °‹,ì ì¶ Œ° (OpenAI°¡ ì ì – í DBì ì ì ¥)"""
        if not self.current_file_id or not self.current_file_data:
            messagebox.showwarning("Œ†‰Œ‡ ", "í ì …ì °¤…ì ì í í ·ì£…ì ,ì .")
            return

        if not self.current_file_data.get("text"):
            messagebox.showwarning("Œ†‰Œ‡ ", "í ·ìƒ °¥… ì ì –í í ì /í ,Œ° ì ì µ° ° /.")
            return

        if not OPENAI_AVAILABLE:
            messagebox.showerror(
                "ì /°¥ ",
                f"OpenAI API°¥… ì ‹ì 'í ì ì ì µ° ° /.\n\n{OPENAI_ERROR or 'OPENAI_API_KEY í Œ
            )
            return

        if not self.tcp_client.is_connected:
            messagebox.showerror("ì /°¥ ", "ì °† ì ì °Œ†°° ì · ì ì§ ì ì µ° ° /.")
            return

        text = self.current_file_data["text"]

        # Œ,°ì¡· í ·ìƒ °“'°¡ Œ° ì ,ì /Œ,° (ì / °‡µ °°'ì§ )
        existing_questions = []
        if hasattr(self.ui, 'quiz_frames') and self.ui.quiz_frames:
            for frame in self.ui.quiz_frames:
                if hasattr(frame, 'question_text'):
                    existing_questions.append({"question": frame.question_text})

        try:
            # í °‹,ì ì ' ì ì –
            self.ui.add_quiz_btn.config(state='disabled', text="ì ì – ì / ...")
            self.ui.root.update()

            quiz_item = generate_single_quiz(text, existing_questions if existing_questions else Non

            # ì °† °¡ ì ì ¡í ì ‹ DBì ì ì ¥
            result = self.tcp_client.send_command("generate_single_quiz", {
                "file_id": self.current_file_id,
                "text": text,
                "question": quiz_item.get("question", ""),
                "answer": quiz_item.get("answer", ""),
                "explanation": quiz_item.get("explanation", "")
            })
```

Client/client_handlers.py

```
            if result.get("status") == "success":
                quiz_id = result.get("quiz_id")
                # UIì   ì   í ·ì ƒ  ì ¶ Œ°
                self.add_quiz_to_ui(quiz_item, quiz_id)

                # current_quiz_data ì  º °ì ·í , (í ·º …ì ·ì ,í , ì ‹ì  ì  í ì  º  ì  ì§ º  º  º¡
                if not self.current_quiz_data:
                    self.current_quiz_data = {"questions": []}
                self.current_quiz_data["questions"].append({
                    "question": quiz_item.get("question", ""),
                    "answer": quiz_item.get("answer", ""),
                    "explanation": quiz_item.get("explanation", ""),
                    "quiz_id": quiz_id
                })

                messagebox.showinfo("ì –Œ‡µ", "í ·ì ƒ Œ°  ì  ì –º  ì · DBì  ì  ì ¥º  ì  ì µº  º /.")
            else:
                messagebox.showerror("ì /º¥ ", f"í ·ì ƒ  ì  ì ¥ ì /í ¤: {result.get('message', '')}")

        except Exception as e:
            import traceback
            traceback.print_exc()
            messagebox.showerror("ì /º¥ ", f"í ·ì ƒ  ì  ì – ì /  ì /º¥ Œ°  º° ì  í  ì µº  º /:\n{e}")
        finally:
            self.ui.add_quiz_btn.config(state='normal', text="â  º /ì  º‹,ì  ì  ì –")

    def build_quiz_ui(self, quiz_data):
        """DBì  ì  º° ì  í ·ì ƒ  º °ì ·í °º¥… Œ,°ì / ì …º¡  UI Œµ‹ì –"""
        # Œ,°ì ¡· í ·ì ƒ  ì  ì fl ì  Œ–° (º† í …Œ‡… Œµ‹º¶ ì  ì  ì  ì§ )
        for child in self.ui.quiz_content.winfo_children():
            if isinstance(child, LabelFrame):
                child.destroy()
        self.ui.quiz_frames.clear()
        self.ui.final_score_label.config(text="")
        self.ui.final_score_label.pack_forget()

        questions = quiz_data.get("questions", [])
        for idx, q in enumerate(questions, start=1):
            quiz_id = q.get("quiz_id")
            self.add_quiz_to_ui(q, quiz_id, idx)

    def add_quiz_to_ui(self, quiz_item, quiz_id=None, index=None):
        """í ·ì ƒ  í  º‹,ì  º¥… UIì  ì ¶ Œ°  (ì ·ì  º µì  í ‹í ¤)"""
        if index is None:
            index = len(self.ui.quiz_frames) + 1

        frame = LabelFrame(self.ui.quiz_content, text=f"º‹,ì  {index}",
                           font=self.ui.heading_font, bg='#ffffff', padx=20, pady=15)
        # add_quiz_btn ì  ì  ì ‰ì
        frame.pack(fill='both', expand=True, pady=10, before=self.ui.add_quiz_btn)

        # º‹,ì  í ì /í , º …º†¤
        question_text = quiz_item.get("question", "")
        question_label = Label(frame, text=question_text, font=self.ui.normal_font,
                               bg='#ffffff', justify='left')
```

```
        question_label.pack(anchor='w', pady=5, fill='x')

        # º‹,ì  í  ì /í ,Œ°  í  º  ì  í ›ì  º§ Œ†  º‡·ì ·º  º¡  wraplength º  ì  ì¡°ì
        def _update_question_wrap(event, label=question_label):
            w = event.width - 50
            if w > 200:
                label.config(wraplength=w)
        frame.bind("<Configure>", _update_question_wrap)

        # º µì  ì  º ¥ì°‰ (ì ·ì  º µì  ì · ì  ì …º'· í  ì  )
        input_box = scrolledtext.ScrolledText(frame, height=8,
                                              font=self.ui.normal_font, wrap='word')
        input_box.pack(fill='both', expand=True, pady=10, padx=5)

        # ì ·ì  º µì  ì · ì  ì …º'· ì  º ¥ì°‰ì  í  ì
        user_answer = quiz_item.get("user_answer", "")
        if user_answer:
            input_box.insert('1.0', user_answer)

        result_label = Label(frame, text="", font=self.ui.normal_font, bg='#ffffff')
        result_label.pack(anchor='w', pady=5)

        # ì  º µ/í ·ì //í ·ì ƒ ID ì  ì ¥
        frame.input_box = input_box
        frame.result_label = result_label
        frame.correct_answer = quiz_item.get("answer", "")
        frame.explanation = quiz_item.get("explanation", "")
        frame.quiz_id = quiz_id
        frame.question_text = question_text

        self.ui.quiz_frames.append(frame)
```